균형 이진 탐색 트리

AVL 트리와 Red-Black 트리

개념 · 시각적 예제 · 삽입과 회전 · 비교

학습 및 팀 발표용

OVERVIEW

전체 학습 흐름

BST의 문제에서 시작해 두 균형 트리의 복구 원리를 연결합니다

1. AVL

BF로 불균형을 찾고
LL·RR·LR·RL 회전으로 복구

2. Red-Black

색, NIL, 블랙 높이로 균형을 제한하고
색 변경과 회전으로 복구

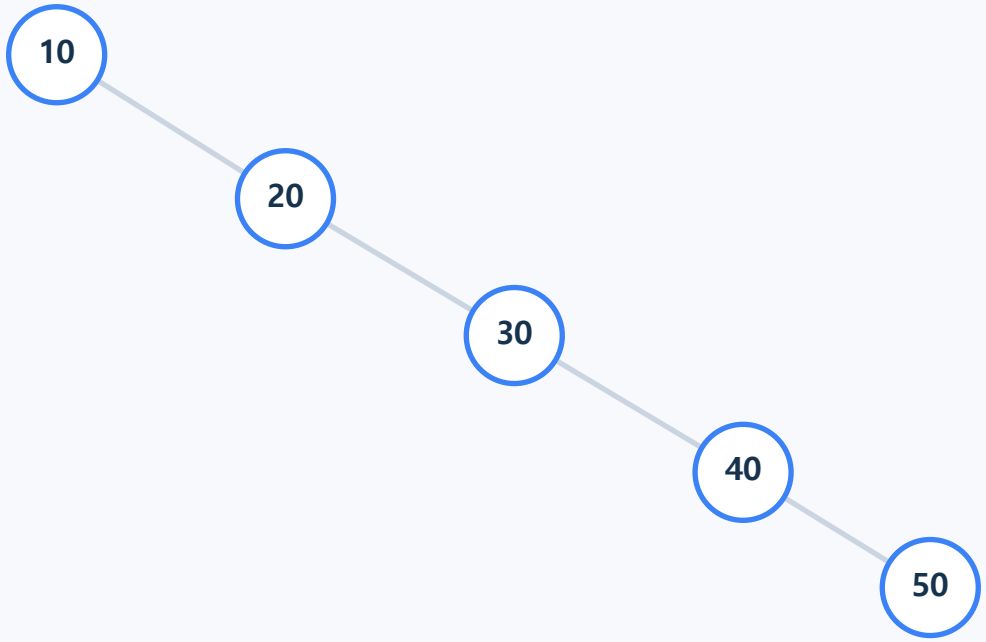
3. 적용

복잡한 삽입 사례를 단계별로 추적하고
두 트리의 특징을 비교

공통 기반 → **BST 대소 관계는 모든 회전 전후에 유지됩니다**

BST가 균형을 필요로 하는 이유

BST 설명은 도입에 필요한 최소 범위만 다룹니다



편향 BST

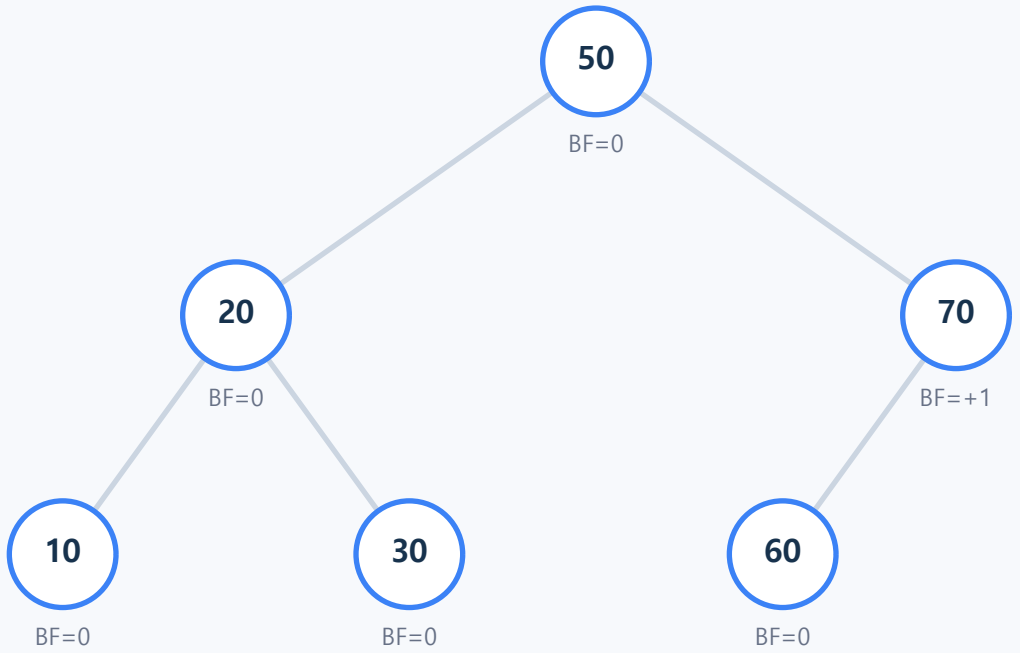
정렬된 값 10, 20, 30, 40, 50을 순서대로 넣으면 한쪽으로 늘어납니다.
탐색·삽입의 최악 시간은 O(n)입니다.

균형 트리의 목표

높이를 O(log n)으로 제한해 탐색·삽입·삭제의 최악 시간을 O(log n)으로 보장합니다.

AVL의 기준: 균형 인수 BF

빈 서브트리 높이 0, 리프 높이 1로 통일합니다



정의

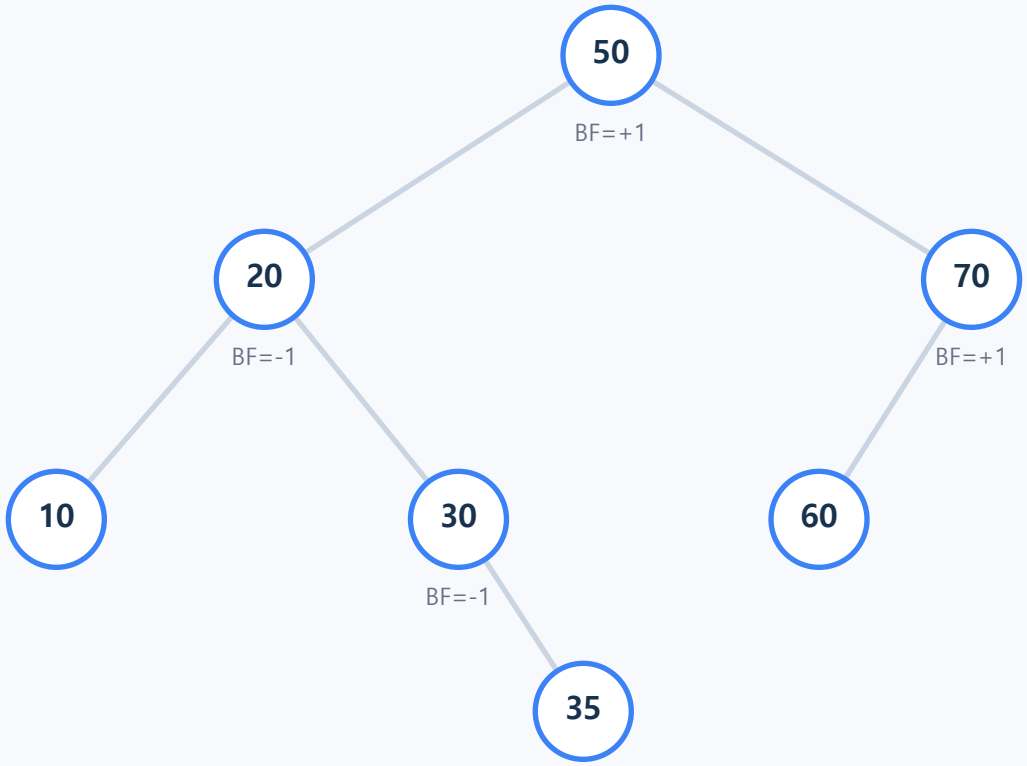
BF(v) = h(왼쪽) - h(오른쪽)

AVL 조건

모든 노드에서 BF ∈ {-1, 0, +1}
노드 수가 아니라 높이 차이를 봅니다.

교정: 35 삽입만으로는 불균형이 아니다

이전 설명의 오류를 바로잡습니다

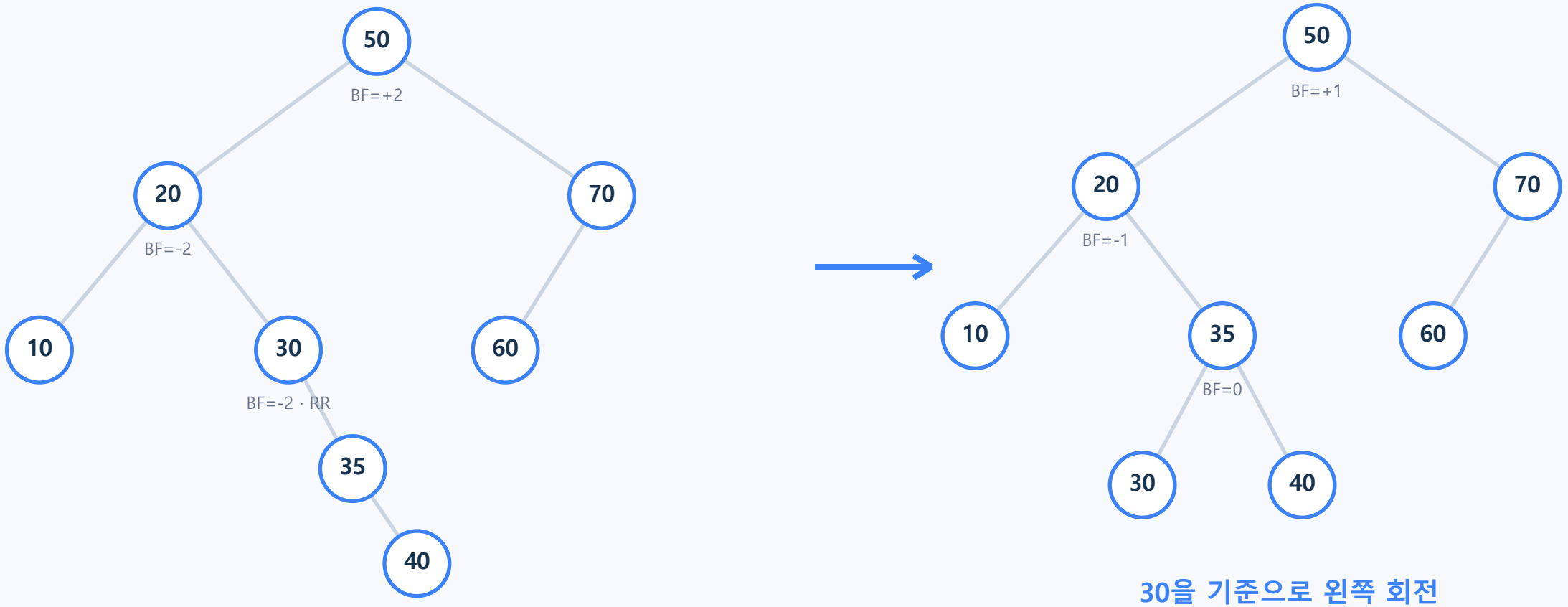


노드	왼쪽 높이	오른쪽 높이	BF
30	0	1	-1
20	1	2	-1
70	1	0	+1
50	3	2	+1

판정
모든 BF가 허용 범위이므로 회전하지 않습니다.

40 삽입에서 처음 불균형이 발생

새 노드에서 위로 올라가며 최초 불균형 노드 30을 찾습니다



30을 기준으로 왼쪽 회전

네 가지 불균형과 회전 선택

이름은 불균형 노드에서 삽입 노드까지의 첫 두 방향을 뜻합니다

LL

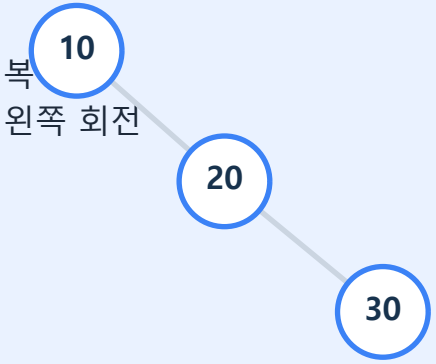
삽입 순서 30·20·10



복구
오른쪽 회전

RR

삽입 순서 10·20·30



복구
왼쪽 회전

LR

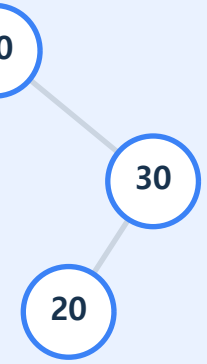
삽입 순서 30·10·20



복구
왼쪽 후 오른쪽

RL

삽입 순서 10·30·20



복구
오른쪽 후 왼쪽

LL: 왼쪽의 왼쪽이 길어짐

삽입 순서: 30, 20, 10



삽입 직후

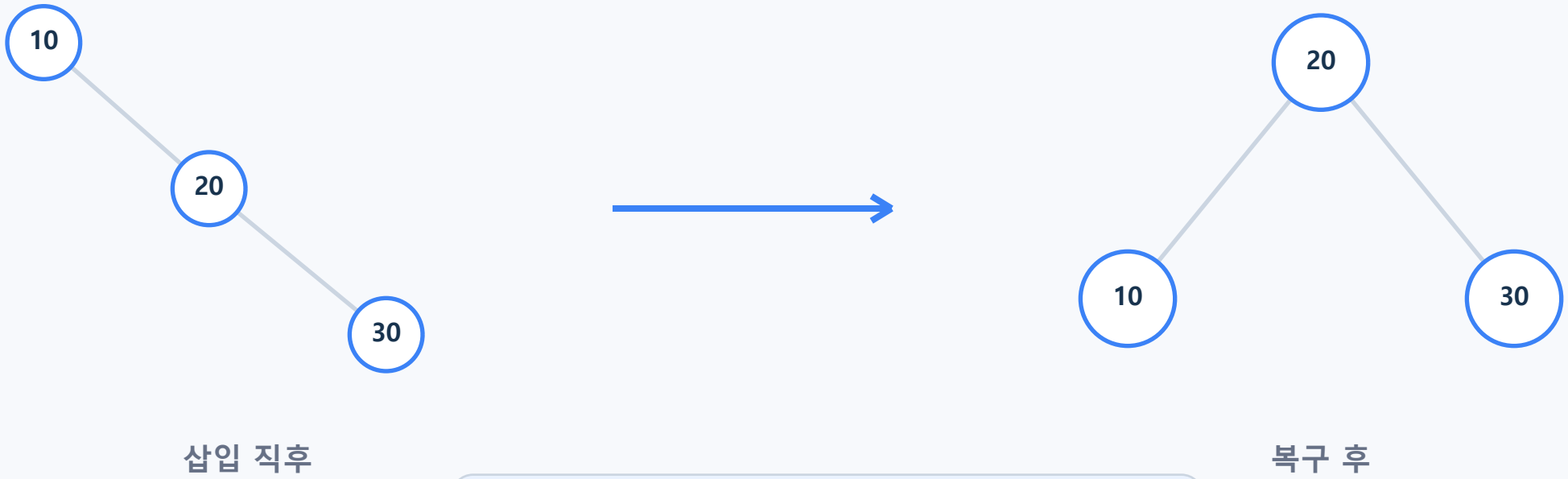
복구 후

회전 규칙

불균형 노드 30에서 오른쪽 회전

RR: 오른쪽의 오른쪽이 길어짐

삽입 순서: 10, 20, 30



삽입 직후

복구 후

회전 규칙

불균형 노드 10에서 왼쪽 회전

LR: 왼쪽의 오른쪽이 길어짐

삽입 순서: 30, 10, 20



삽입 직후

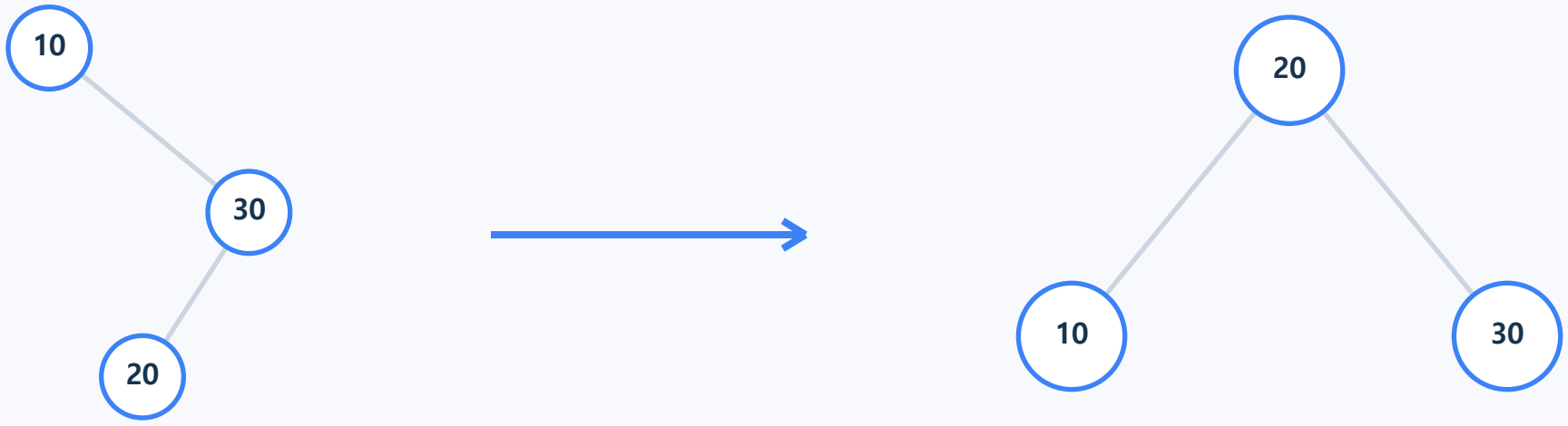
복구 후

회전 규칙

10에서 왼쪽 회전 후 30에서 오른쪽 회전

RL: 오른쪽의 왼쪽이 길어짐

삽입 순서: 10, 30, 20



삽입 직후

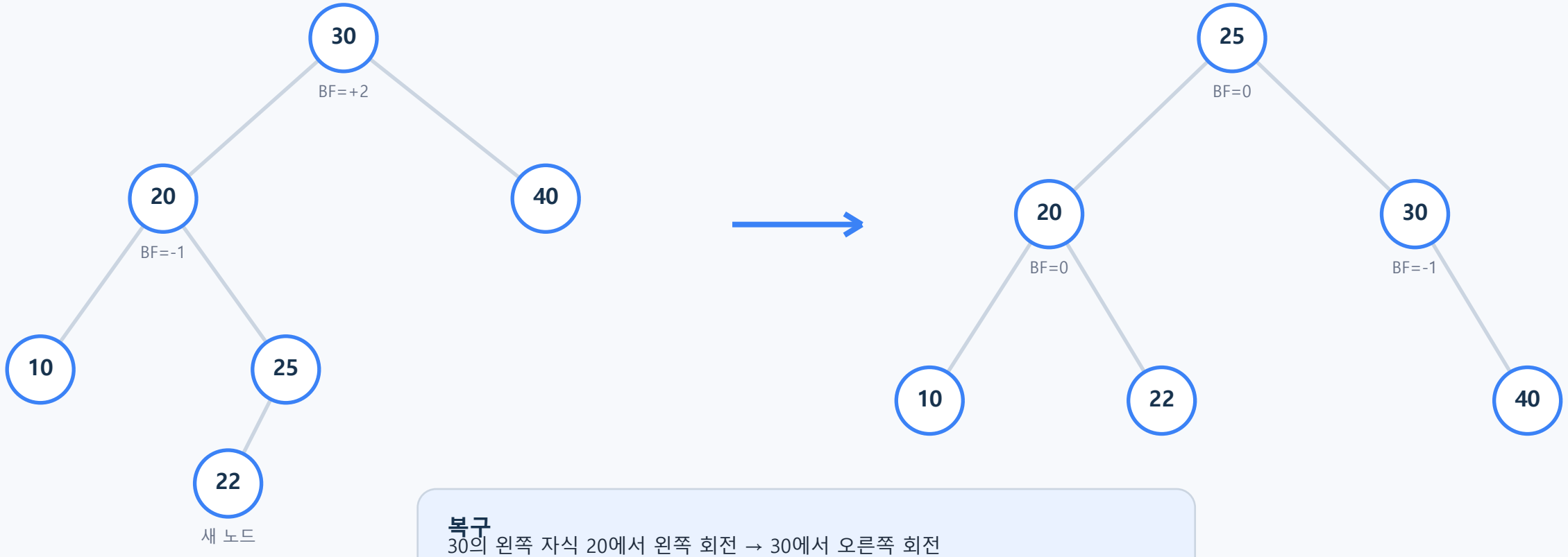
복구 후

회전 규칙

30에서 오른쪽 회전 후 10에서 왼쪽 회전

복잡한 AVL 삽입: 22가 만드는 LR

초기 트리 30, 20, 40, 10, 25에 22를 삽입합니다



복구
30의 왼쪽 자식 20에서 왼쪽 회전 → 30에서 오른쪽 회전

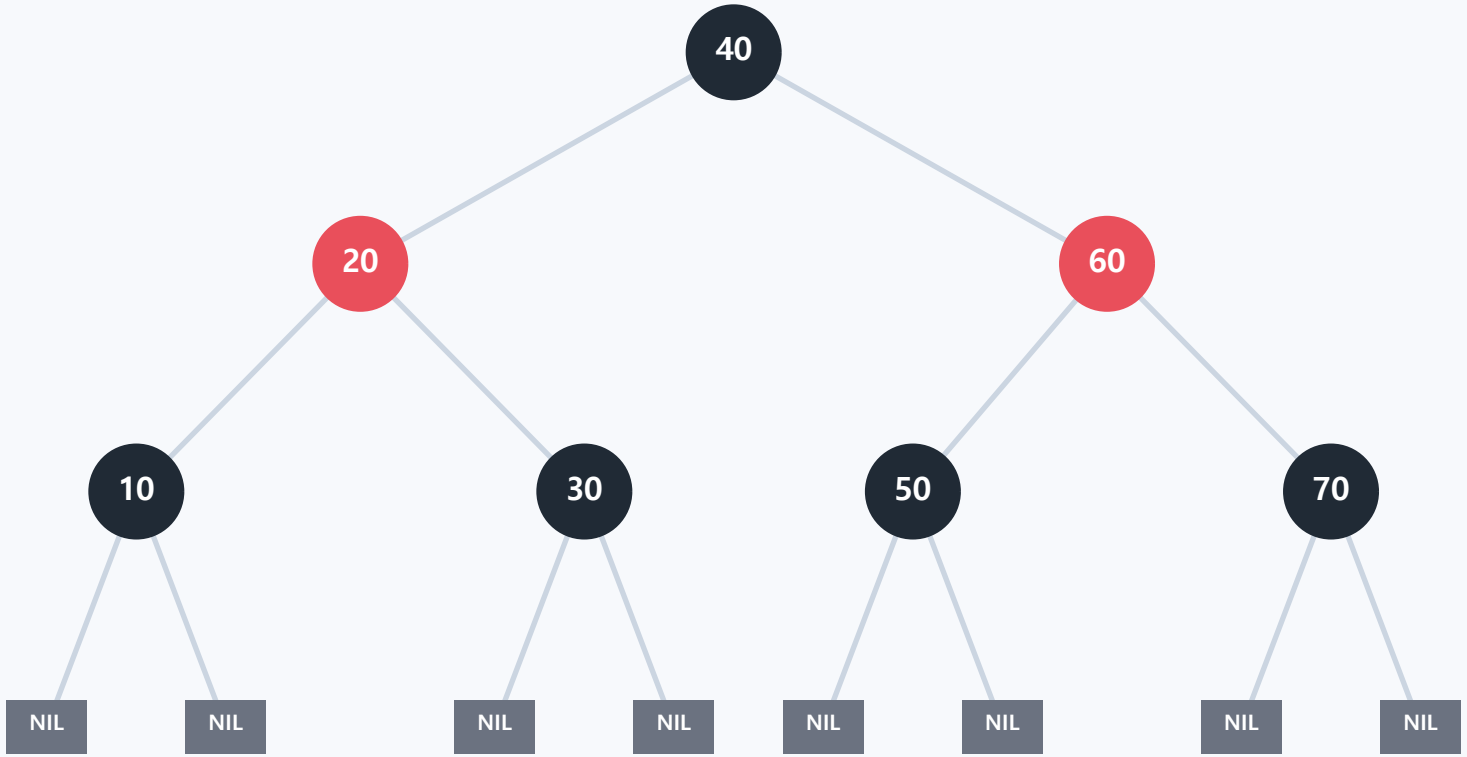
Red-Black 트리의 5가지 속성

BST 대소 관계 위에 색과 경로 규칙을 추가합니다

1. 모든 노드는 RED 또는 BLACK
2. 루트는 BLACK
3. 모든 NIL 리프는 BLACK
4. **RED 노드의 자식은 모두 BLACK**
5. **한 노드에서 하위 NIL까지 모든 경로의 BLACK 수가 같음**

NIL과 블랙 높이

bh(x)는 보통 x를 제외하고 NIL을 포함해 셉니다



경로 예

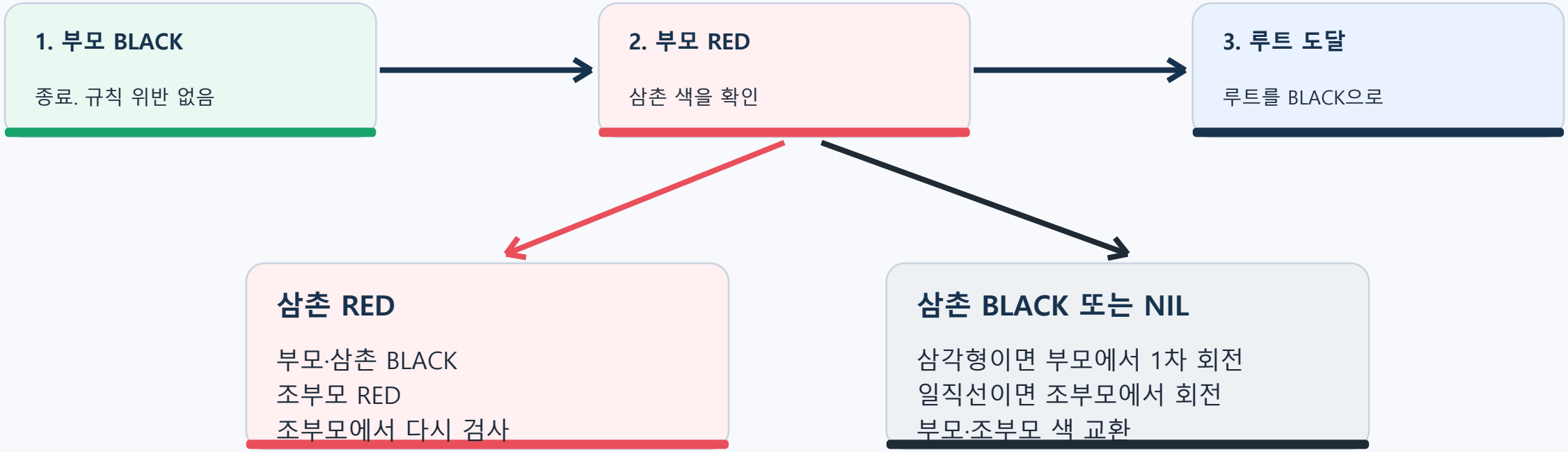
40 → 20 → 10 → NIL
루트 포함 BLACK 수 = 3
bh(40) = 2

왜 균형인가

모든 루트-NIL 경로의 BLACK 수가 같습니다.
RED는 연속할 수 없습니다.

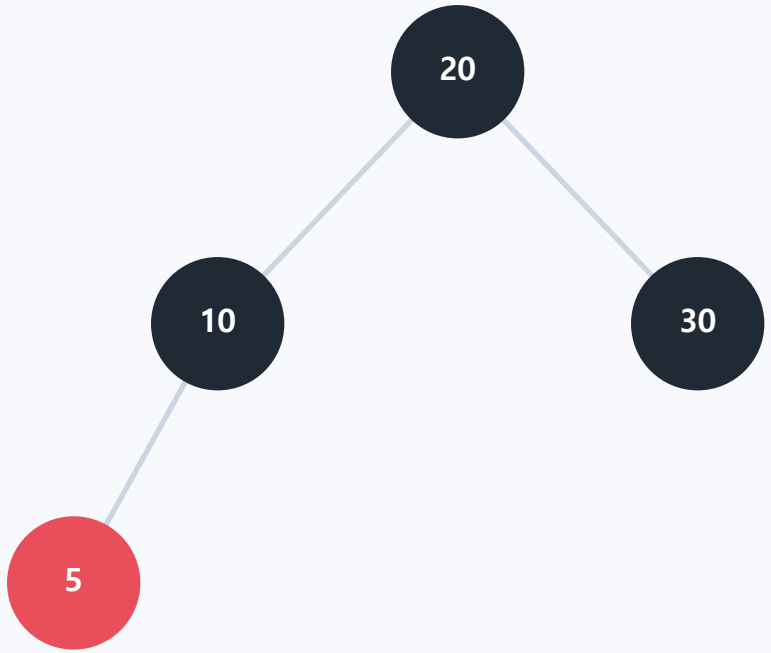
삽입 복구의 판단 순서

새 노드는 RED로 넣어 블랙 높이를 즉시 바꾸지 않습니다



부모 BLACK: 삽입 즉시 종료

새 노드 5는 RED, 부모 10은 BLACK

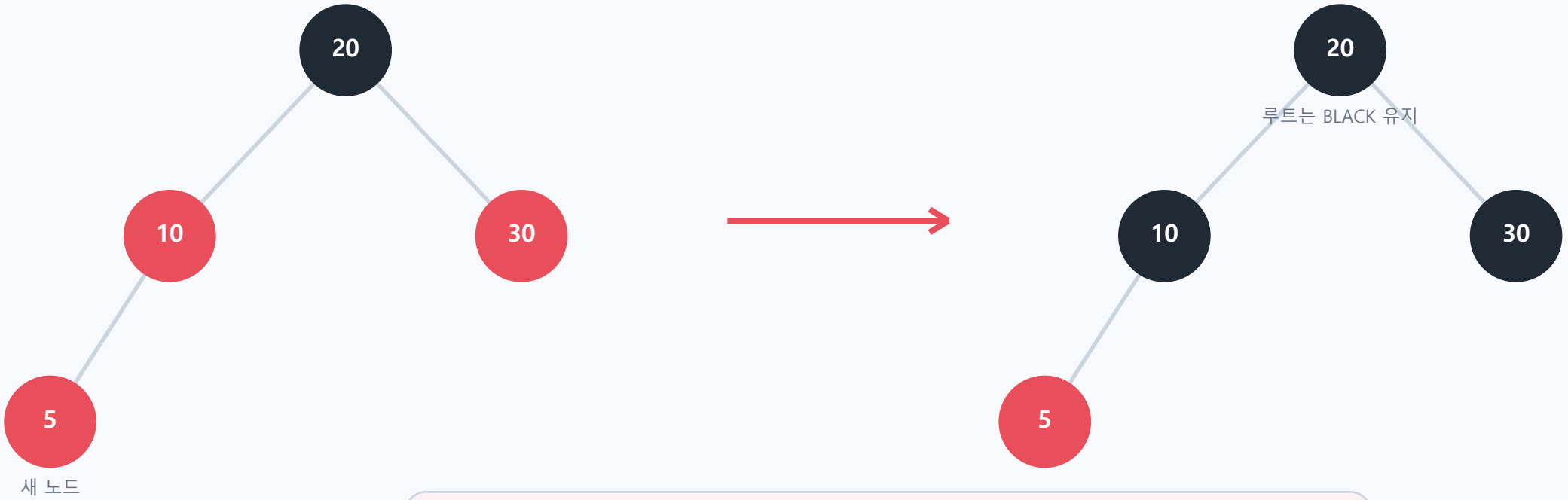


검사 결과

RED-RED 연속이 없습니다.
새 노드가 RED라 블랙 높이도 그대로입니다.
색 변경과 회전 모두 필요 없습니다.

삼촌 RED: 색 변경 후 위로 전파

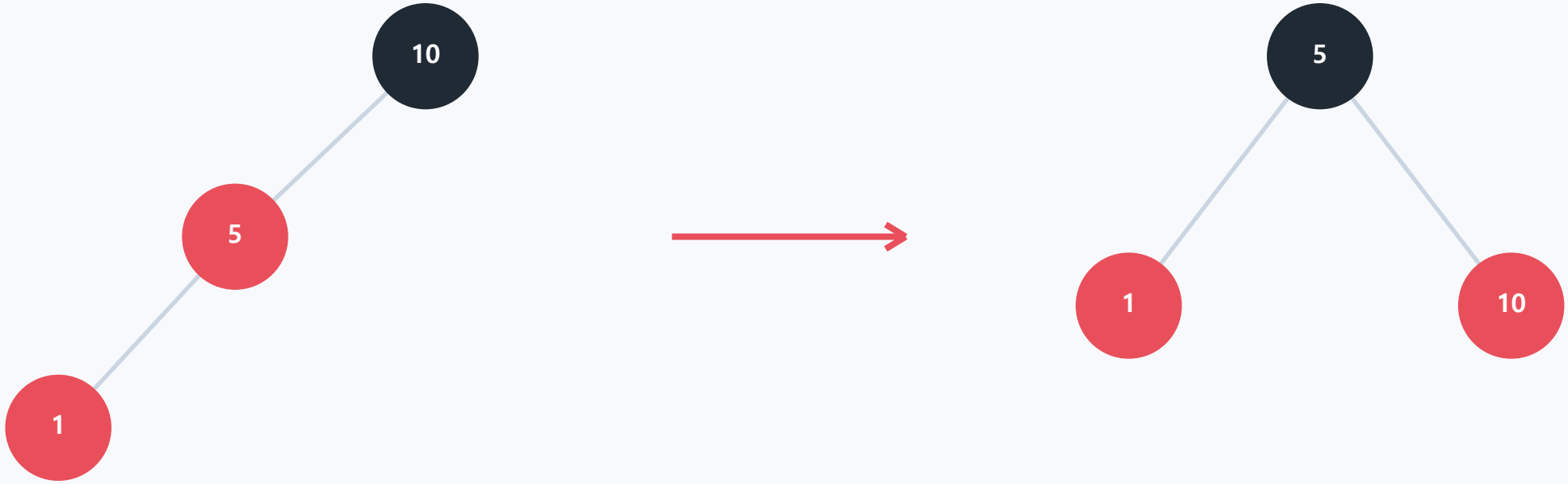
20(B)의 자식 10(R), 30(R) 아래에 5(R)를 삽입



복구
부모 10과 삼촌 30을 BLACK, 조부모 20을 RED로 바꾼 뒤 루트이므로 다시 BLACK

삼촌 BLACK, 일직선: 한 번 회전

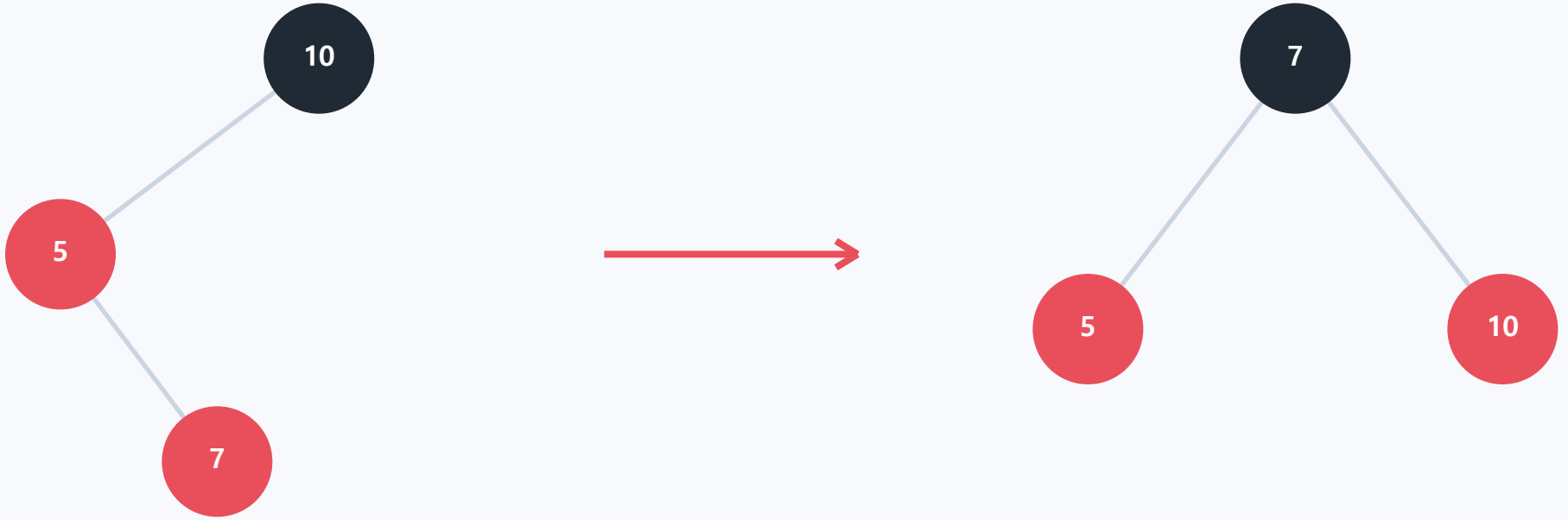
10(B) → 5(R) → 1(R), 삼촌은 NIL(B)



복구
조부모 10에서 오른쪽 회전, 부모 5와 조부모 10의 색을 교환

삼촌 BLACK, 삼각형: 두 번 회전

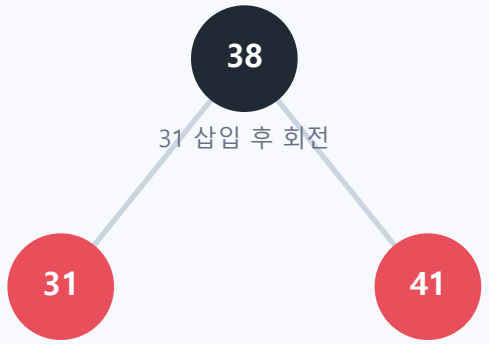
10(B) → 5(R)의 오른쪽에 7(R) 삽입



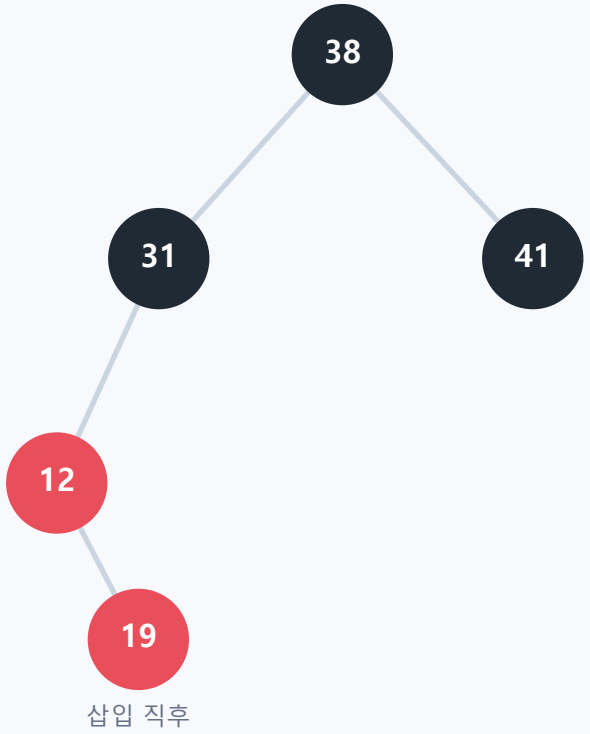
복구
부모 5에서 왼쪽 회전해 일직선으로 만든 뒤, 조부모 10에서 오른쪽 회전하고 색 교환

Red-Black 복합 삽입: 41, 38, 31, 12, 19, 8

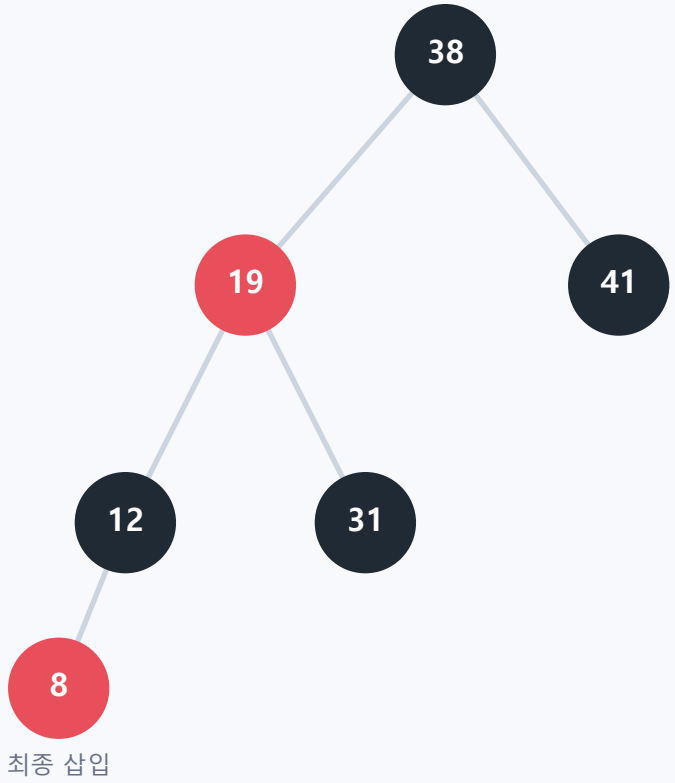
대표 순서를 세 장면으로 압축해 색 변경과 회전의 연쇄를 확인합니다



① 31: 일직선 회전



② 12: 삼촌 RED 색 변경
③ 19: 삼각형 두 번 회전



④ 8: 삼촌 RED 색 변경
모든 경로 bh(38)=2

COMPARISON

AVL과 Red-Black 비교

둘 다 BST 순서를 유지하며 최악 O(log n)을 보장합니다

항목	AVL	Red-Black
균형 기준	모든 노드의 |BF| ≤ 1	색 규칙과 동일한 블랙 높이
균형 강도	더 엄격함	조금 더 유연함
탐색	낮은 높이로 유리한 편	최악 O(log n)
삽입·삭제	회전 빈도가 높은 편	색 변경으로 끝나는 경우가 많음
대표 활용	탐색 비중이 큰 구조	표준 라이브러리의 맵·셋 계열

선택 기준: 읽기 중심이면 AVL의 엄격한 균형, 갱신이 잦으면 Red-Black의 유연한 복구를 고려

SUMMARY

발표 마무리

두 트리는 같은 목표를 서로 다른 규칙으로 달성합니다

AVL

높이 차이를 직접 측정
BF로 최초 불균형을 찾음
네 가지 회전으로 즉시 복구

Red-Black

색과 블랙 높이로 편향 제한
부모·삼촌·조부모를 확인
색 변경 또는 회전으로 복구

회전은 값의 순서를 바꾸지 않고 부모-자식 관계만 바꿉니다

검산 습관: AVL은 모든 BF, Red-Black은 RED-RED와 모든 NIL 경로의 BLACK 수